

Informe Presentación 2 Testing Software

Christian Ossa Rol: 201873565-5

Javier Perez Riveros Rol: 201873517-5

9 de diciembre de 2022

Link Github: <https://github.com/Software-testing-course-projects/Reacetario>

1. Descripción de Trabajo Realizado

Teniendo ya cómo base el Proyecto del Recetario, se procedió a realizar todo el trabajo de Backend para poder almacenar las recetas cómo fue solicitado, esto sumado a tener que cambiar el Frontend de la pagina para que se integre con el nuevo Backend. Esto fue realizado en su totalidad por Javier, quien fue el encargado de esta área del trabajo, siendo revisado y analizado también por Christian quien, tras realizada y montada la pagina, procedió al desarrollo del testing.

Para dar más detalle al proceso, se procedió a lero crear el entorno de integración continua, creando la organización en Github, la sala en Slack y colocar las incidencias en JIRA. Esto para luego proceder a instalar las dependencias necesarias para la creación del Backend, utilizando Docker para poder montar todo el lado del servidor y, eventualmente, para evitar problemas que se dieron durante el desarrollo, se subió todo a AWS.

Se realizó la integración con Jenkins, se dejó hecho todo para crear el Pipeline y finalmente se procedió al lado de Testing. Durante el tiempo que se realizó la integración de Front con Back y las herramientas solicitadas, se estuvo estudiando el cómo realizar el testing con Selenium, para que una vez llegada esta instancia se tomara más tiempo realizando el testing que aprendiendo a cómo hacerlo.

Una vez hecho el testing, se procedió a ajustar todo para que el pipeline funcionase, la pagina estuviera subida donde debía, la pagina pasara todos los testings y finalmente hacer los merges finales a main para hacer uso del método de trabajo de Gitflow.

Hablando un poco más del metodo de trabajo, se trabajó en dos ramas protegidas llamadas “main” y “dev”. Estas ramas son protegidas, lo que significa que solo se pueden hacer cambios en ellas mediante un pull request. Además, se trabajó en otras ramas llamadas “Feature”, “Release” y “Fix”, que se mergearon primero a la rama “dev” y luego a la rama “main”.

Los pull request de “dev” se configuraron como “Squash and Merge” y los de “main” como “Merge Commits”, con el objetivo de tener un orden en el repositorio. Cada pull request se configuró para requerir la revisión y aprobación de un desarrollador distinto al que creó la rama.

A continuación se presenta un grafo del proyecto, donde se puede observar que la rama “main” solo recibió merge commits de la rama “dev” y esta última solo recibió merge commits de las ramas “Feature”, “Release” y “Fix”.

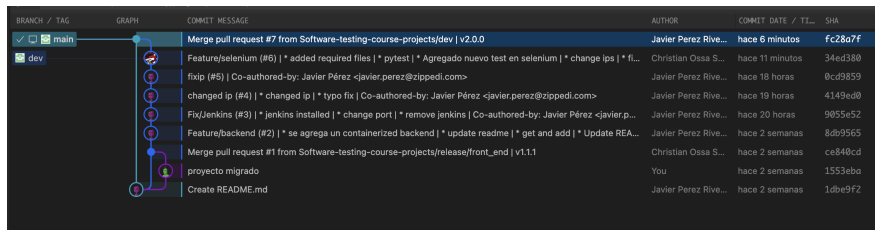


Figura 1: Captura de grafo de proyecto

Esto se ajustó al procedimiento de trabajo en fechas que se estableció en la carta Gantt, aunque cómo esto tenía primordialmente un enfoque más ágil, algunas cosas se tardaron más de lo usual debido a los problemas técnicos que se presentaron.

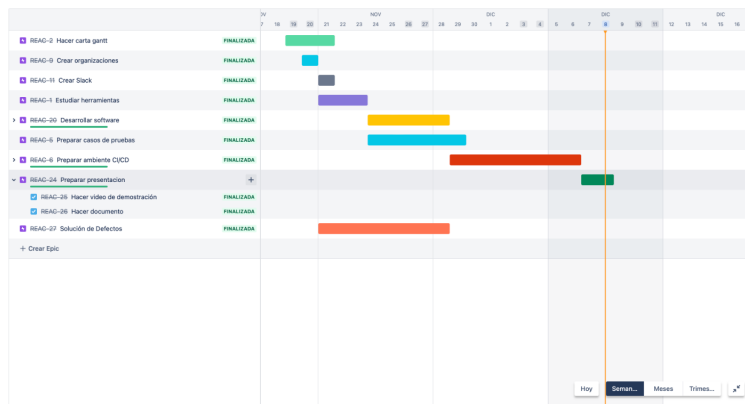


Figura 2: Captura de la hoja de ruta

2. Construcción de Pruebas

Dado que el método de pruebas eran pruebas automatizadas en un entorno de integración continua, se utilizó Selenium cómo fue solicitado para realizar el testing de las interfaces. Para poder realizar el testing se realizaron los casos de prueba que iban a ser tomados en cuenta, esto para poder tener más orden y tomando en cuenta lo ya realizado en la pagina con el Backend integrado.

Nombre	Agregar Receta			UC01
Resumen	Se agrega una receta a la pagina, añadiendole imagen, descripción y unos pasos dummy.			
Actores	Pagina, Programador (Este último solo interactúa para este testing)			
Pre-Condiciones	No debe haber una receta con el mismo nombre de la que se va a añadir.			
Flujos	#	Flujo Principal	#	Flujo Alternativo
	1	Se abre una tab de google chrome con Webdriver		
	2	Se busca el botón de añadir receta		
	3	Se añade título, descripción, imagen y demás campos		
	4	Se verifica que se ha añadido, confirmando la imagen añadida		
	5	Si la imagen coincide, se toma la prueba cómo pasada.	4.1	Si la imagen no coincide, se toma la prueba cómo fallida
Post-Condiciones	Existe ahora una nueva receta en la pagina, llamada "Completo Italiano".			
Nombre	Editar Receta			UC02
Resumen	Se toma una receta ya creada y se le editan los campos.			
Actores	Pagina, Programador (Este último solo interactúa para este testing)			
Pre-Condiciones	Debe haber una receta con el título "Completo Italiano"			
Flujos	#	Flujo Principal	#	Flujo Alternativo
	1	Se abre una tab de google chrome con Webdriver		
	2	Se busca y se hace match con la receta con el título correcto.		
	3	Se edita título, imagen, intreridentes y pasos.		
	4	Se verifica la edición, viendo si la imagen nueva está.		
	5	Si la imagen coincide, se toma la prueba cómo pasada.	4.1	Si la imagen no coincide, se toma la prueba cómo fallida
Post-Condiciones	La receta antes llamada "Completo Italiano", cambió sus datos y ahora se llama "HotDog Italiano".			
Nombre	Eliminar Receta			UC03
Resumen	Se toma una receta ya creada y se elimina			
Actores	Pagina, Programador (Este último solo interactúa para este testing)			
Pre-Condiciones	Debe haber una receta con el título "HotDog Italiano"			
Flujos	#	Flujo Principal	#	Flujo Alternativo
	1	Se abre una tab de google chrome con Webdriver		
	2	Se busca y se hace match con la receta con el título correcto.		
	3	Se busca y se abre el botón de "más" y luego se presiona el de "borrar"		
	4	Se verifica la eliminación de la tarjeta, buscando su título.		
	5	Si el nombre no está en la lista de tarjetas, se pasa la prueba.	4.1	Si el nombre está en la lista de tarjetas, se toma la prueba cómo fallida.
Post-Condiciones	Ya no existe la receta agregada y editada antes llamada "Completo Italiano".			

Figura 3: Casos de Uso

Se tomó en cuenta un CRUD sencillo para el testing, o sea simplemente Añadir, Editar y Eliminar las recetas (esto sumado al visualización que viene implícito). Por ende se crearon estos 3 casos de uso, estos se harán en un testing con Selenium y con Pytest para poder verificar cada test hecho (además de realizarlo). Esto se hizo adicionalmente en un Pipeline en Jenkins que se verá más adelante para automatizar todo.

2.1. Añadido de Receta

```
@pytest.fixture()
def add_testing(testSetup):
    # Clickear en un elemento, usando ID.
    try:
        driver.find_element(By.ID, "Addbutton").click()

        # Testing Add Receta

        driver.find_element(By.ID, "titulo").send_keys("Completo Italiano" + Keys.ENTER)
        driver.find_element(By.ID, "descripcion").send_keys("Completo" + Keys.ENTER)
        driver.find_element(By.ID, "image").send_keys("https://upload.wikimedia.org/wikipedia/commons/e/e0/completo_italiano.jpg" + Keys.ENTER)
        driver.find_element(By.NAME, "ingrediente").send_keys("Un completo" + Keys.ENTER)
        driver.find_element(By.NAME, "paso").send_keys("Hacer un completo" + Keys.ENTER)
        driver.find_element(By.ID, "AddRecetabutton").click()

        time.sleep(2)
    except Exception as e:
        print(e)
        assert False
```

Figura 4: Testing de Añadir Receta

```
def test_add(add_testing):
    try:
        time.sleep(5)
        element = driver.find_elements(By.CLASS_NAME, "Tarjeta")
        found = False
        for tarjeta in element:
            if "Completo Italiano" in tarjeta.text:
                found = True
                imagen = tarjeta.find_element(By.CLASS_NAME, "Imagen")
                assert imagen.get_attribute("src") == "https://upload.wikimedia.org/wikipedia/commons/e/e0/completo_italiano.jpg"
                time.sleep(2)
                break
        if not found:
            assert False
    except Exception as e:
        print(e)
        assert False
```

Figura 5: Verificación de Añadir Receta

Para el Testing se utiliza Selenium para añadir cosas a la pagina y así simular una entrada real, el 1er codigo muestra esto, añadiendo un 'Completo Italiano', luego en el 2do codigo, se utiliza Pytest (aunque en ambos técnicamente se utiliza) para assertar o no una verificación de lo realizado en la 1era función, básicamente la 1era función realiza visualmente el testing y la 2da función lo verifica en terminal. Cabe mencionar que el backend está hecho para que no se pueda añadir otra receta con el mismo nombre.

2.2. Edición de Receta

```
@pytest.fixture()
def edit_testing(testSetup):
    try:
        time.sleep(5)

        elements = driver.find_elements(By.CLASS_NAME, "Tarjeta")
        for tarjeta in elements:
            if "Completo Italiano" in tarjeta.text:
                tarjeta.find_element(By.ID, "MasButton").click()

                time.sleep(2)

                menu = driver.find_element(By.ID, "basic-menu")

                time.sleep(2)

                menu.find_element(By.ID, "EditarBoton").click()

                time.sleep(4)
                driver.find_element(By.ID, "titulo").send_keys(Keys.SHIFT + Keys.UP, Keys.BACKSPACE)
                driver.find_element(By.ID, "titulo").send_keys("HotDog Italiano" + Keys.ENTER)
                driver.find_element(By.ID, "descripcion").send_keys(Keys.SHIFT + Keys.ARROW_UP)
                driver.find_element(By.ID, "descripcion").send_keys(Keys.BACKSPACE)
                driver.find_element(By.ID, "descripcion").send_keys("HotDog" + Keys.ENTER)
                driver.find_element(By.ID, "descripcion").send_keys(Keys.BACKSPACE)
                driver.find_element(By.ID, "descripcion").send_keys(Keys.ENTER)
```

Figura 6: Testing de Editar Receta

```
def test_edit(edit_testing):
    try:
        time.sleep(5)
        element = driver.find_elements(By.CLASS_NAME, "Tarjeta")
        found = False

        for tarjeta in element:
            if "HotDog Italiano" in tarjeta.text:
                found = True
                imagen = tarjeta.find_element(By.CLASS_NAME, "imagen")
                assert imagen.get_attribute("src") == "https://static.onecms.io/wp-content/uploads/sites/19/2017/06/05/elcompleto.jpg"
                time.sleep(2)
                break
        if not found:
            assert False
    except Exception as e:
        print(e)
        assert False
```

Figura 7: Verificación de Editar Receta

Para la función de testeo de la edición el código es más largo, pero sigue los mismos lineamientos de la imagen, esto aprieta los botones necesarios para entrar a la ventana que permite editar una tarjeta en particular y finalmente le cambia el título y las cosas. La 2da función hace la misma función de verificar por terminar que se logró cambiar algo exitosamente. Aquí debe haber una tarjeta creada anteriormente con el título “HotDog Italiano”.

2.3. Eliminación de Receta

```
@pytest.fixture()
def delete_testing(testSetup):
    try:
        time.sleep(5)

        elements = driver.find_elements(By.CLASS_NAME, "Tarjeta")
        for tarjeta in elements:
            if "Hotdog italiano" in tarjeta.text:
                tarjeta.find_element(By.ID, "MasButton").click()

                time.sleep(2)

                menu = driver.find_element(By.ID, "basic-menu")

                time.sleep(2)

                menu.find_element(By.ID, "BorrarBoton").click()

                time.sleep(4)
    except Exception as e:
        print(e)
        assert False
```

Figura 8: Testing de Eliminación de Receta

```
def test_delete(delete_testing):
    try:
        time.sleep(5)
        element = driver.find_elements(By.CLASS_NAME, "Tarjeta")

        for tarjeta in element:
            assert "Hotdog italiano" not in tarjeta.text
            time.sleep(2)
    except Exception as e:
        print(e)
        assert False
```

Figura 9: Verificación de Eliminación de Receta

Para este último test se busca el botón de 'mas' de la tarjeta deseada y finalmente se elimina, esto se verifica buscando el titulo de la receta editada anteriormente en el set de recetas, si no está verifica la prueba cómo pasada.

Se adjunta adicionalmente una captura de las pruebas pasadas en terminal.

```
C:\Windows\System32\cmd.exe
C:\Users\rages\Desktop\Docs_Generales\Uni_Docs\Testing Software\Proyecto_Recetas\Reacetario>pytest -v -s test.py
===== test session starts =====
platform win32 -- Python 3.10.5, pytest-7.2.0, pluggy-1.0.0 -- C:\Users\rages\AppData\Local\Programs\Python\Python310\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\rages\Desktop\Docs_Generales\Uni_Docs\Testing Software\Proyecto_Recetas\Reacetario
collected 3 items

test.py::test_add PASSED
test.py::test_edit PASSED
test.py::test_delete PASSED
```

Figura 10: Visualización de Todas las pruebas Pasadas.

3. Pipeline

Se realizó un pipeline en Jenkins que consta de cuatro fases: “GIT”, “INSTALL DEPENDENCIES”, “RUN TEST” y “POST”. En la fase “GIT”, se clona el proyecto y se cambia a la rama requerida. En la fase “INSTALL DEPENDENCIES”, se instalan las librerías necesarias para ejecutar las pruebas. En la fase “RUN TEST”, se ejecuta el script de pruebas realizado con Selenium+Pytest. Finalmente, en la fase “POST”, se procesan los resultados generados por Pytest y se utilizan para mostrar en Jenkins cuántos tests pasaron, fallaron o tuvieron errores.

Cada vez que se escribía un nuevo test, se ejecutaba el pipeline para verificar que estuviera bien escrito. Mientras se escribían nuevos tests y se modificaban partes de la página, estas pruebas ayudaban a asegurar que los nuevos cambios no afectaban negativamente la estabilidad de la página. De esta manera, se mantenía un alto grado de calidad y se evitaban errores o fallos en la página.

Es importante mencionar que, con el fin de lograr un trabajo en equipo durante el proyecto, se expuso el puerto 8080 para permitir el acceso remoto a Jenkins. De esta manera, ambos desarrolladores pudieron acceder a Jenkins desde cualquier lugar y no solo a la página alojada en AWS. Esto facilitó la colaboración y permitió una mayor eficiencia en el desarrollo del proyecto.

El pipeline se encuentra en la figura 11 y el resultado de las pruebas se encuentran en la figura 12.


```

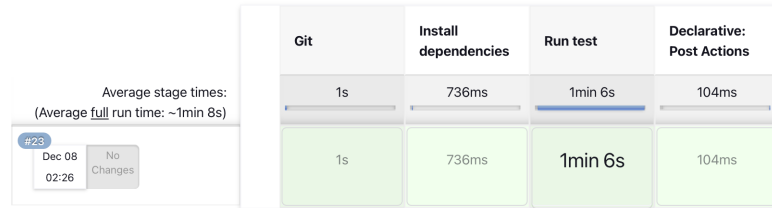
pipeline {
    agent any

    stages {
        stage("Git"){
            steps{
                script{
                    checkout([ $class: 'GitSCM', branches:
                        [[name: '*/main']],
                    userRemoteConfigs: [[ url: 'https://github.com/
                        Software-testing-course-projects/Reacetario.git'
                    ]]])
                }
            }
        }
        stage("Install dependencies"){
            steps{
                script{
                    sh 'pip3 install -r requirements.txt'
                }
            }
        }
        stage("Run test"){
            steps{
                script{
                    sh 'python3 -m pytest -v -s test.py
                        --junit-xml=pytest_unit.xml'
                }
            }
        }
    }
    post {
        always {
            junit 'pytest_unit.xml'
        }
    }
}

```

Figura 11: Pipeline

Stage View



Resultado de tests : test

0 fallidos (±0)

3 tests (±0)
Tardó 2 Min 12 Seg.
[añadir descripción](#)

Todos los tests

Nombre del test	Duración	Estado
test_add	44 Seg	Pasados
test_delete	36 Seg	Pasados
test_edit	51 Seg	Pasados

Tendencia de los resultados de pruebas

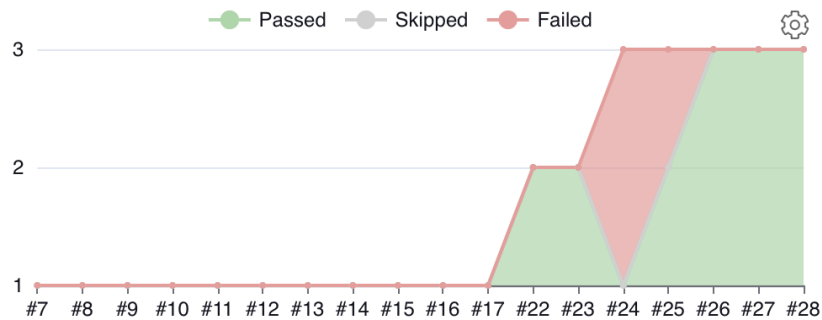


Figura 12: Resultados de los test realizados

4. Conclusiones

En conclusion, se realizó el trabajo de Backend para almacenar las recetas, se modificó el Frontend de la página para integrarlo con el nuevo Backend y se llevó a cabo el testing con Selenium. Además, se creó un entorno de integración

continúa utilizando Github, Slack y JIRA, y se utilizó Docker para montar el lado del servidor. Todo el proceso fue llevado a cabo por Javier y revisado por Christian, siguiendo el método de trabajo de Gitflow y utilizando ramas protegidas para realizar los cambios mediante pull request. Los resultados del testing se procesaron mediante un pipeline en Jenkins. En general, se trabajó de manera eficiente y organizada para completar el proyecto del Recetario.

Después de completar el proyecto del Recetario, se puede decir que se trabajó de manera eficiente y organizada, siguiendo el método de trabajo de Gitflow y utilizando ramas protegidas para realizar los cambios mediante pull request. El uso de herramientas como Jenkins, Docker, Jira y Selenium facilitaron el desarrollo y permitieron llevar a cabo un testing automatizado en un entorno de integración continua.

Aunque se enfrentaron algunos desafíos durante el proceso, como problemas con Docker, se logró superarlos trabajando con herramientas de tipo Cloud como lo es AWS EC2 y completar el proyecto de manera satisfactoria. Además, el trabajo en equipo fue fundamental para avanzar en el proyecto y lograr una buena coordinación entre los desarrolladores.

En general, se puede decir que se adquirieron nuevos conocimientos y habilidades durante el desarrollo del proyecto, y se logró crear una página web funcional y de alta calidad.